주문·재고 운영 관리

정합성에서 이벤트 전달, 인증과 배포까지 연결한 관리 시스템

Admin Portfolio

주문 한 건이 저장된 뒤에도
재고·이벤트·통계가 이어지도록 설계했습니다.

프로젝트 목적

단순 CRUD를 출발점으로 동시 재고 변경, 외부 메시지 전송 실패, 중복 이벤트와 인증·권한 문제를 다뤘습니다. React 화면으로 업무 흐름을 확인하고 AWS에 실제 배포했습니다.

구현 범위

Spring Boot API · MySQL 스키마와 조회 · Redis 세션/캐시/요청 제한 · Kafka/Outbox · Keycloak OIDC · React 관리 화면 · Jenkins와 AWS 배포

읽는 순서

구성도 → 업무 정합성 → Outbox와 멱등 처리 → 인증/Redis → 배포·검증 → 실제 화면

실행 서비스 https://13-124-81-142.sslip.io

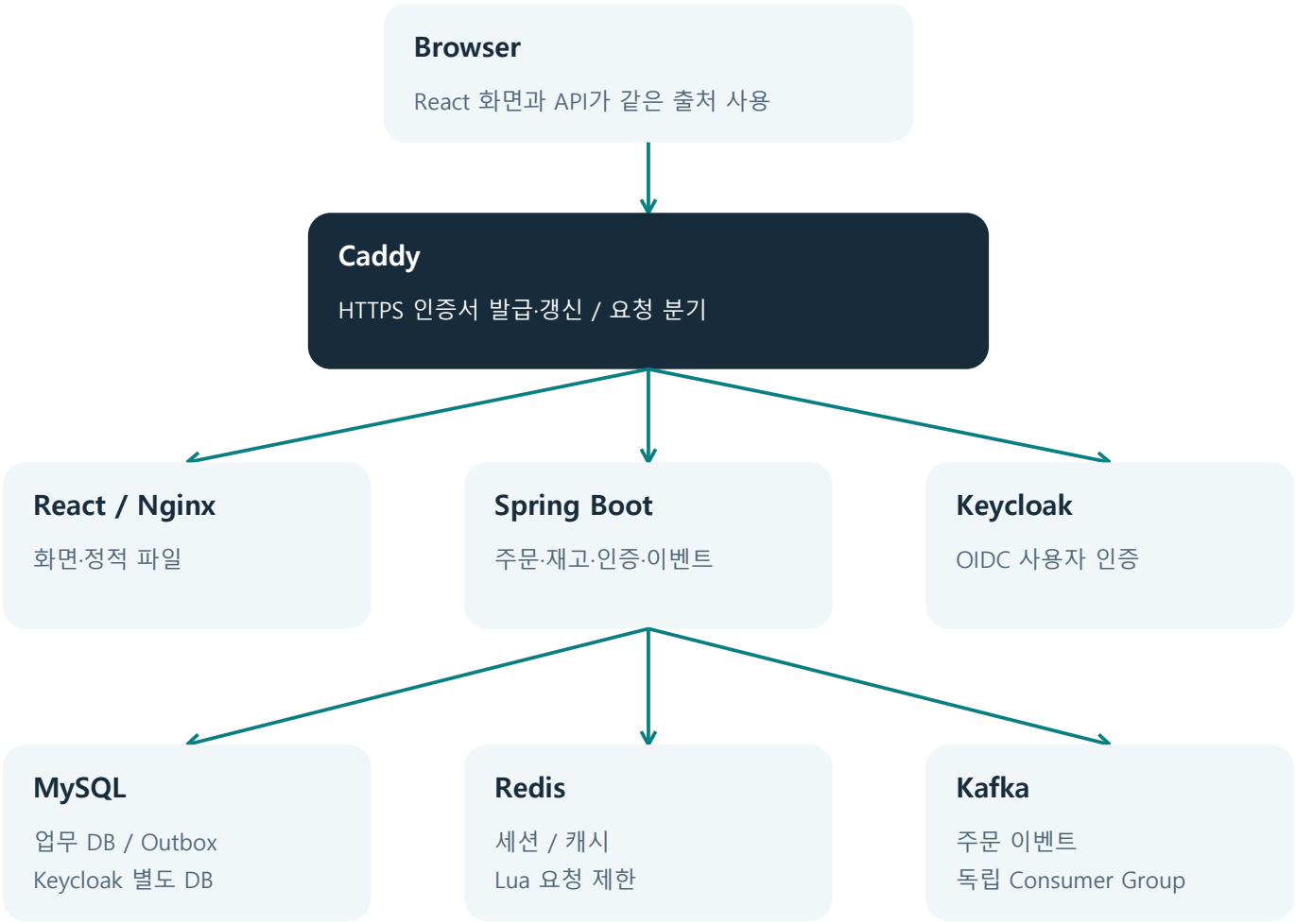
Backend github.com/whdcks0101-beep/admin-api

Frontend github.com/whdcks0101-beep/admin-frontend

로그인은 사전 등록 방식입니다. 관리자 비밀번호는 공개하지 않습니다.

HTTPS 주소 하나로 연결

AWS Lightsail / Ubuntu / Docker Compose



Spring Boot는 Keycloak과 토큰을 교환하고, Kafka 이벤트를 소비해 MySQL 통계를 갱신합니다. 위 화살표는 주요 연결을 요약합니다.

클라우드 실행 경계

백엔드 한 인스턴스와 인프라를 단일 Lightsail 서버에서 실행합니다. Caddy만 공개 HTTP/HTTPS 포트를 사용하고 DB·Redis·Kafka는 내부 통신으로 연결합니다.

로컬 Kubernetes 경험

별도 Docker Desktop 환경에서 백엔드 2개 Pod, Redis 세션 공유, Pod 재생성, 상태 점검, 롤링 업데이트와 롤백을 실습했습니다.

재고 충돌과 부분 저장 방지

문제 → 트랜잭션 경계 → 충돌 처리 → 검증

문제

두 요청이 같은 수량을 읽고 각각 저장하면 한 변경이 사라질 수 있습니다. 다품목 주문 중 일부 재고만 부족한 경우에도 주문과 차감 내역이 함께 취소되어야 합니다.

선택한 구현

Inventory에 JPA @Version을 적용해 오래된 버전의 변경을 거부합니다. OrderService.createOrder의 쓰기 트랜잭션에 주문·주문 항목·재고 차감·Outbox 저장을 함께 포함했습니다.

하나의 DB 트랜잭션

주문 저장 → 재고 차감 → 주문 항목 저장 → Outbox 저장
모두 커밋하거나 모두 롤백

사용자에게 보이는 동작

재고 부족은 주문 실패로 안내합니다. 낙관적 락 충돌은 HTTP 409로 전달하고, 사용자가 최신 상태를 조회해 다시 판단하도록 합니다.

조회와 저장의 역할

QueryDSL로 주문 번호·기간 조건을 조합하고 페이징을 제공합니다. 상품 연관 데이터가 필요한 조회에서는 fetch join을 사용합니다. MySQL 전용 선점·멱등 저장 쿼리는 native SQL로 구현했습니다.

검증 근거

InventoryOptimisticLockTest, OrderServiceRollbackTest, OrderOutboxTransactionTest로 충돌·전체 롤백·동시 저장을 검증합니다. AWS 화면에서 주문 생성과 재고 차감도 확인했습니다.

전송 실패와 중복 전달에 대응

Transactional Outbox + 소유권 기반 선점 + 멱등 소비

왜 Outbox인가

주문 DB 커밋 직후 Kafka 전송에 실패해도 다시 처리할 이벤트가 DB에 남도록 했습니다. 주문 이벤트를 동일 트랜잭션에 저장하고 Relay가 나중에 전송합니다.

PENDING
재시도 시각 도달

→

PROCESSING
토큰·선점 만료 저장

→

PUBLISHED
전송 후 완료 기록

여러 실행자가 동시에 처리할 때

FOR UPDATE SKIP LOCKED로 작업을 선점합니다. DB 잠금은 짧은 선점 트랜잭션에서만 유지하고 전송 중에는 해제합니다. 완료·실패 기록은 claim token과 만료 시각을 확인하며 DB 시간을 사용합니다.

중복 전송을 전제로 통계 보호

전송 성공 후 완료 기록에 실패하면 이벤트가 다시 전달될 수 있습니다. order_statistics의 order_id 고유 키와 ON DUPLICATE KEY UPDATE로 동일 주문의 통계 중복 저장을 방지합니다.

소비 실패를 추적할 수 있도록

업무 처리 실패는 재시도 후 DLT로 이동합니다. 역직렬화에 실패한 JSON은 원문 바이트를 보존합니다. 로그와 통계는 서로 다른 Consumer Group에서 처리합니다.

검증과 해석

동일 주문 재처리 시 통계 건수가 유지됨을 확인했습니다. 이 설계는 전송 자체의 exactly-once 보장이 아니라, 재전송 가능한 전달과 멱등 소비의 조합입니다.

인증과 서비스 권한을 분리

Keycloak OIDC / Spring Security / Redis

SSO 연결

Keycloak은 사용자 본인 인증을 담당합니다. Spring Security가 issuer + subject로 연결된 서비스 회원을 찾고, 우리 DB의 회원 구분으로 권한을 부여합니다. 이메일 일치만으로 관리자 계정을 자동 연결하지 않습니다.

Keycloak 인증 → 회원 연결 조회 → 서비스 권한 적용

사전 등록 방식 / 미연결 계정 거부 / SSO 로그아웃 확인

세션과 접근 제어

Spring Session Redis로 인스턴스 간 세션을 공유합니다. 변경 요청에는 CSRF 보호를 적용하고 인증 실패와 권한 부족은 API에서 401/403으로 구분합니다. AWS 세션 쿠키에 Secure·HttpOnly를 적용했습니다.

상품 캐시와 무효화

상품 조회 캐시를 사용하고 수정 시 캐시 삭제 요청을 Outbox에 저장합니다. 편집용 조회는 DB를 직접 읽습니다. 재고처럼 최신 상태 판단이 필요한 업무와 캐시 사용 범위를 구분했습니다.

요청 횟수 제한

Redis Lua로 카운터 변경과 만료 처리를 원자적으로 실행합니다. 여러 인스턴스가 같은 제한을 공유하며 초과 요청은 429로 안내합니다. 이 기능은 같은 주문 요청의 중복 생성을 막는 멱등성 키와 별개입니다.

확인한 결과

AWS에서 일반 로그인, SSO 로그인, 관리자 메뉴 접근, SSO 로그아웃을 확인했습니다. 로컬에서는 Redis 세션 공유와 요청 제한, 캐시 무효화를 검증했습니다.

버튼 한 번으로 AWS 배포

Jenkins는 개발 PC에서, 서비스는 AWS에서 실행

01 GitHub checkout

백엔드·프론트 저장소 분리

02 테스트와 이미지 빌드

백엔드 테스트 / 프론트 린트·테스트

03 SSH 이미지 전송

서버 공개키 검증 / Jenkins Secret file

04 컨테이너 교체·상태 확인

배포 잠금 / 성공한 이미지 태그만 기록

추적 가능한 배포

aws-빌드번호-Git커밋 태그를 사용합니다. frontend와 backend를 각각 배포하고 서버 잠금으로 변경을 직렬화합니다. 단일 컨테이너 교체 중에는 잠시 연결이 끊길 수 있으며 실패 시 자동 롤백하지 않습니다.

테스트 실행 기록

2026-10-26 로컬 기본 테스트: 79개 중 60개 통과, 19개 조건부 건너뜀, 실패 0개. MySQL·실제 Redis 통합 테스트는 별도 활성화 대상입니다. 프론트 빌드에는 린트와 테스트 단계가 포함됩니다.

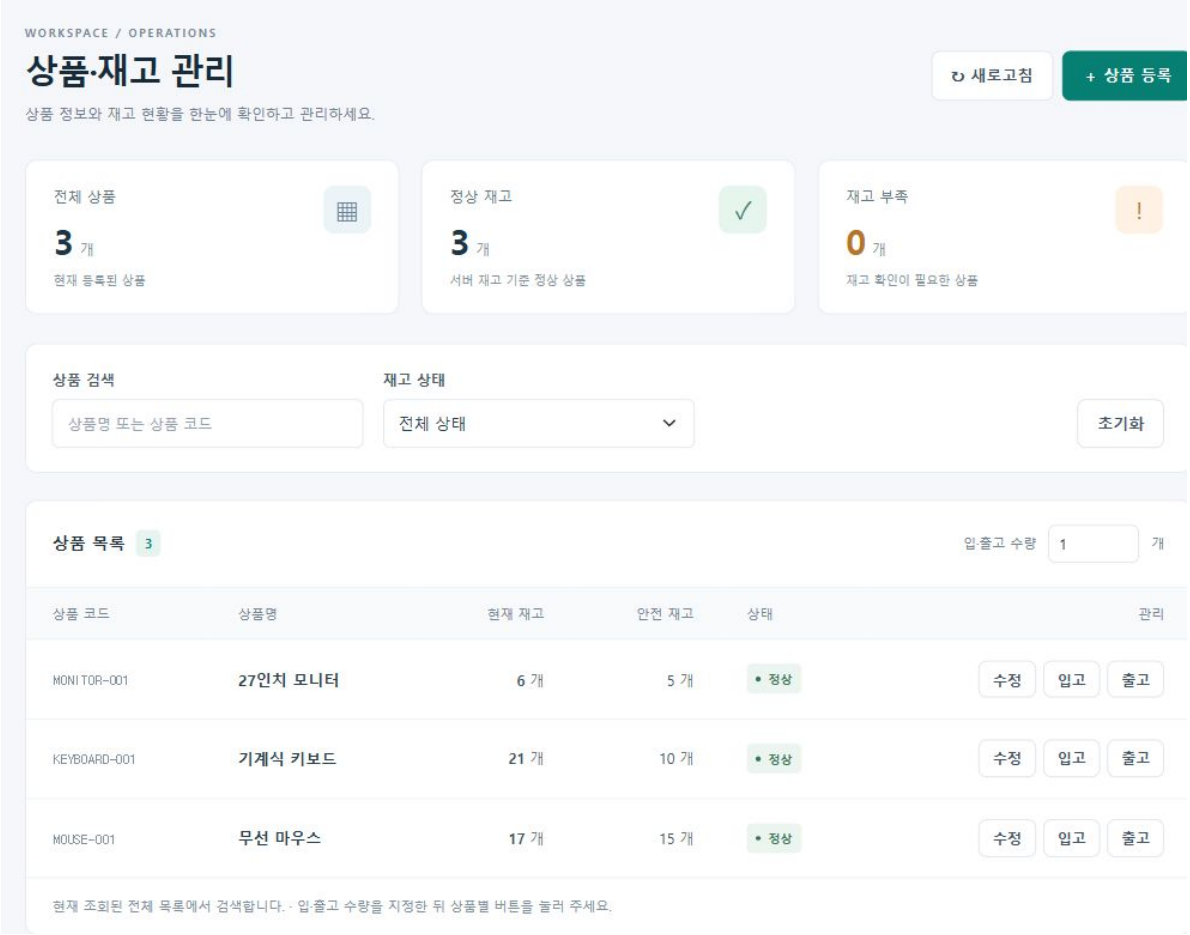
클라우드 사용자 검증

백엔드·프론트 Jenkins 작업 성공 후 실제 로그인·목록 조회를 확인했습니다. 주문 생성, 재고 차감, Kafka를 통한 날짜별 통계 반영과 SSO 로그아웃도 확인했습니다.

상품·재고에서 주문 상세까지

2026-10-26 AWS 서비스 캡처 / 반응형 화면 / 샘플 데이터

상품·재고 관리

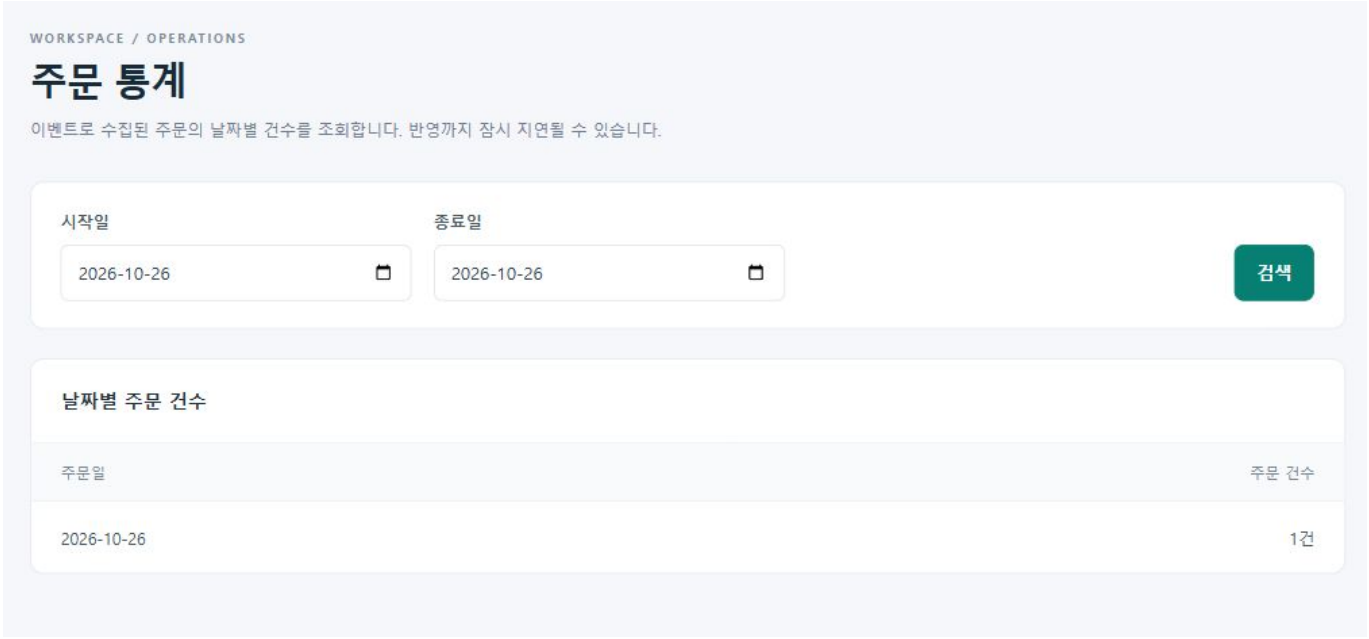


주문 상세



이벤트가 통계로 이어지는 결과

주문 생성 후 비동기로 반영된 일별 주문 건수



확인한 업무 흐름

주문·재고·Outbox 커밋 → Relay → Kafka → 통계 Consumer
→ 일별 주문 건수 조회

화면에서 확인한 결과

2026-10-26 주문 1건이 통계에 반영된 실제 조회 결과입니다. 비동기 처리이므로 주문 저장 직후에는 잠시 지연될 수 있습니다.

구현 사례와 소스: GitHub backend README

실행 서비스: https://13-124-81-142.sslip.io